Igor Fernando Casita Ferreira da Silva

Comparação Empírica entre Quicksort e Radix Sort: Fundamentos Teóricos, Metodologia Experimental e Análise Estatística de Desempenho

São Paulo

2026

Igor Fernando Casita Ferreira da Silva

Comparação Empírica entre Quicksort e Radix Sort: Fundamentos Teóricos, Metodologia Experimental e Análise Estatística de Desempenho

Relatório técnico apresentado à disciplina de Introdução à Análise de Algoritmos (IAA), do curso de Bacharelado em Sistemas de Informação da Escola de Artes, Ciências e Humanidades da Universidade de São Paulo (EACH-USP), como parte dos requisitos de avaliação do Exercício-Programa 2 (EP2).

Universidade de São Paulo – USP

Escola de Artes, Ciências e Humanidades – EACH

Bacharelado em Sistemas de Informação

Orientador: Prof. Dr. Fabio Nakano

São Paulo

2026

RESUMO

Este relatório técnico apresenta uma comparação empírica entre os algoritmos de ordenação *quicksort* e *radix sort* (variante *least-significant-digit-first*, base 2^8), desenvolvida no âmbito do Exercício-Programa 2 (EP2) da disciplina de Introdução à Análise de Algoritmos (IAA) do curso de Sistemas de Informação da EACH-USP. Após a fundamentação teórica das complexidades temporal e espacial de ambos os algoritmos, implementações em linguagem C foram submetidas a testes empíricos de desempenho em uma máquina física (Intel Core i7-1355U, 16 GB de RAM, Windows 11), medindo-se o tempo de execução por meio das funções `clock()` e `gettimeofday()` para tamanhos de entrada variando de 10^3 a 2×10^9 elementos. Os dados coletados foram analisados estatisticamente com o software R, por meio de regressão log-log para estimar os expoentes empíricos de crescimento e do teste não paramétrico de Wilcoxon para amostras pareadas. Os resultados confirmam que o radix sort apresenta desempenho superior ao quicksort na ampla maioria dos tamanhos testados (expoente empírico $\hat{k}_{\text{radix}} \approx 1{,}02$ contra $\hat{k}_{\text{quicksort}} \approx 1{,}06$; $p \approx 7{,}8 \times 10^{-13}$), com uma inversão notável em $n = 2 \times 10^9$, atribuída à pressão de memória decorrente do espaço auxiliar $O(n + b)$ do radix sort em um ambiente com memória RAM limitada. Discutem-se também as limitações impostas pelo modelo de dados LLP64 do Windows, que restringiu o teste de tamanhos de entrada superiores a $2{,}147 \times 10^9$ elementos.

Palavras-chave: Algoritmos de ordenação. Quicksort. Radix sort. Análise de complexidade. Benchmarking.

ABSTRACT

This technical report presents an empirical comparison between the quicksort and radix sort (least-significant-digit-first variant, base 2^8) sorting algorithms, developed for Programming Exercise 2 (EP2) of the Introduction to Algorithm Analysis (IAA) course at the Information Systems program of EACH-USP. After establishing the theoretical foundations of the time and space complexity of both algorithms, C-language implementations were empirically benchmarked on physical hardware (Intel Core i7-1355U, 16 GB RAM, Windows 11), measuring execution time with the `clock()` and `gettimeofday()` functions for input sizes ranging from 10^3 to 2×10^9 elements. The collected data were statistically analyzed using R, through log-log regression to estimate empirical growth exponents and the non-parametric paired Wilcoxon signed-rank test. Results confirm that radix sort outperforms quicksort for the large majority of tested sizes (empirical exponent $\hat{k}_{\text{radix}} \approx 1.02$ versus $\hat{k}_{\text{quicksort}} \approx 1.06$; $p \approx 7.8 \times 10^{-13}$), with a notable inversion at $n = 2 \times 10^9$, attributed to memory pressure caused by radix sort's $O(n + b)$ auxiliary space requirement under constrained RAM. The limitations imposed by Windows' LLP64 data model, which restricted testing of input sizes above 2.147×10^9 elements, are also discussed.

Keywords: Sorting algorithms. Quicksort. Radix sort. Complexity analysis. Benchmarking.

LISTA DE ILUSTRAÇÕES

Figura 1 – Tempo de execução em função do tamanho de entrada (escala log-log) . 18
Figura 2 – Razão entre o tempo do quicksort e do radix sort, por tamanho de entrada 18
Figura 3 – Distribuição dos tempos de execução por algoritmo (*boxplot*) 20

LISTA DE TABELAS

Tabela 1 – Comparativo teórico entre quicksort e radix sort 14
Tabela 2 – Ambiente de hardware e software utilizado nos experimentos 15
Tabela 3 – Tempo médio de execução por tamanho de entrada (gettimeofday, segundos) 17
Tabela 4 – Resultados da regressão log-log: $\log(\bar{t}) = \log(c) + \hat{k}\log(n)$ 19

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
CLRS	Cormen, Leiserson, Rivest e Stein (autores de referência em algoritmos)
EACH	Escola de Artes, Ciências e Humanidades
EP	Exercício-Programa
IAA	Introdução à Análise de Algoritmos
LCG	*Linear Congruential Generator* (gerador congruente linear)
LSD	*Least Significant Digit* (dígito menos significativo)
RAM	*Random Access Memory*
USP	Universidade de São Paulo

LISTA DE SÍMBOLOS

$O(\cdot)$	Limitante assintótico superior (*big-O*)
$\Omega(\cdot)$	Limitante assintótico inferior
$\Theta(\cdot)$	Limitante assintótico justo (*tight bound*)
n	Número de elementos do vetor de entrada
b	Base numérica utilizada no radix sort ($b = 256$)
d	Número de dígitos/passadas do radix sort
$\hat{k}$	Expoente empírico de crescimento estimado por regressão log-log

SUMÁRIO

Introdução **10**

1 FUNDAMENTAÇÃO TEÓRICA **11**
1.1 Quicksort **11**
1.1.1 Descrição do algoritmo 11
1.1.2 Complexidade temporal 11
1.1.3 Complexidade espacial 12
1.1.4 Estabilidade 12
1.2 Radix Sort **12**
1.2.1 Descrição do algoritmo 12
1.2.2 Complexidade temporal 13
1.2.3 Complexidade espacial 14
1.2.4 Estabilidade 14
1.3 Síntese comparativa **14**

2 METODOLOGIA **15**
2.1 Ambiente de execução **15**
2.2 Instrumentação de tempo **15**
2.3 Protocolo experimental **16**
2.4 Análise estatística **16**

3 RESULTADOS E DISCUSSÃO **17**
3.1 Resultados agregados **17**
3.2 Regressão log-log **19**
3.3 Teste de Wilcoxon pareado **19**
3.4 A inversão em $n = 2 \times 10^9$: pressão de memória **20**
3.5 Tamanhos de entrada não testados **21**

4 RESPOSTAS AO QUESTIONÁRIO **22**
4.1 Questão 1 . . . . . **22**
4.2 Questão 2 . . . . . **22**
4.3 Questão 3 . . . . . **22**
4.4 Questão 4 . . . . . **22**
4.5 Questão 5 . . . . . **22**

5 CONCLUSÃO **23**

REFERÊNCIAS 24

APÊNDICES 25

APÊNDICE A – CÓDIGO-FONTE DAS IMPLEMENTAÇÕES . . . 26
A.1 quicksort.c 26
A.2 radixsort.c 28
A.3 rng.c 30
A.4 elapsedRunningTime.c 31

APÊNDICE B – DADOS BRUTOS 33

INTRODUÇÃO

Ordenar um conjunto de dados é uma das operações mais recorrentes em computação, utilizada direta ou indiretamente por buscas binárias, algoritmos de otimização, bancos de dados e sistemas de recuperação de informação. A literatura clássica (CORMEN et al., 2009) descreve dezenas de algoritmos de ordenação com diferentes garantias assintóticas, premissas sobre o domínio das chaves e padrões de acesso à memória, não havendo um algoritmo universalmente superior para todos os cenários.

Este Exercício-Programa 2 (EP2) da disciplina de Introdução à Análise de Algoritmos (IAA), do curso de Bacharelado em Sistemas de Informação da EACH-USP, tem por objetivo comparar, de forma teórica e empírica, dois algoritmos de ordenação com estratégias fundamentalmente distintas:

- **Quicksort**: um algoritmo de ordenação por comparação, baseado em particionamento recursivo, com complexidade média $\Theta(n \log n)$;
- **Radix sort**: um algoritmo de ordenação não baseado em comparação, baseado em contagem por dígitos, com complexidade $\Theta(d \cdot (n + b))$, que na prática se comporta como $\Theta(n)$ para o domínio de inteiros de 32 bits aqui considerado.

O relatório está organizado da seguinte forma: o Capítulo 1 apresenta a fundamentação teórica dos dois algoritmos, incluindo prova da complexidade média do quicksort e justificativa de por que o radix sort escapa do limite inferior $\Omega(n \log n)$ válido para algoritmos de ordenação por comparação. O Capítulo 2 descreve o ambiente de execução, as ferramentas de medição de tempo e a metodologia estatística empregada. O Capítulo 3 apresenta e discute os resultados experimentais, incluindo a inversão de desempenho observada em $n = 2 \times 10^9$. O Capítulo 4 responde ao questionário do enunciado do EP2, e o Capítulo 5 traz as considerações finais.

1 FUNDAMENTAÇÃO TEÓRICA

1.1 QUICKSORT

1.1.1 DESCRIÇÃO DO ALGORITMO

O quicksort é um algoritmo de ordenação por comparação que segue o paradigma de *dividir para conquistar* (CORMEN et al., 2009). Dado um vetor $A[p..r]$, o algoritmo:

1. escolhe um elemento *pivô*;
2. particiona o vetor em duas regiões, de modo que todos os elementos à esquerda do pivô sejam menores ou iguais a ele, e todos à direita sejam maiores;
3. aplica-se recursivamente o mesmo procedimento às duas subpartições.

A implementação utilizada neste trabalho adota as seguintes otimizações práticas, usuais em implementações de produção (CORMEN et al., 2009):

- **Seleção de pivô por mediana de três**: reduz a probabilidade de escolha de pivôs extremos em entradas adversárias ou já ordenadas;
- **Partição de Hoare**: dois ponteiros avançam das extremidades em direção ao centro, reduzindo o número de trocas em relação ao esquema de Lomuto;
- **Corte para *insertion sort*** em subvetores de tamanho $k \leq 16$: para entradas pequenas, o custo constante do quicksort recursivo supera o de um algoritmo simples como o *insertion sort*, que possui melhor desempenho de cache e menor sobrecarga;
- **Eliminação de recursão de cauda**: a maior das duas subpartições é tratada iterativamente (laço `while`), e apenas a menor é processada por chamada recursiva, garantindo profundidade de pilha $O(\log n)$ mesmo no caso médio.

1.1.2 COMPLEXIDADE TEMPORAL

Teorema 1.1 (Complexidade média do quicksort)**.** *O número esperado de comparações realizadas pelo quicksort sobre um vetor de n elementos distintos, em posições iniciais aleatórias, é* $O(n \log n)$.

Esboço da prova (variáveis indicadoras). Sejam $z_1 < z_2 < \cdots < z_n$ os elementos do vetor ordenado. Defina a variável indicadora $X_{ij} = 1$ se z_i e z_j são comparados em algum momento da execução, e $X_{ij} = 0$ caso contrário. O número total de comparações é

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}.$$

Observa-se que z_i e z_j são comparados se, e somente se, um deles é escolhido como pivô antes de qualquer elemento estritamente entre eles. Para um conjunto de $j-i+1$ elementos (de z_i a z_j), a probabilidade de que z_i ou z_j seja o primeiro pivô escolhido entre eles é $\dfrac{2}{j-i+1}$, assumindo pivôs uniformemente aleatórios. Logo,

$$\mathbb{E}[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{2}{j-i+1} = O(n \log n),$$

pois a soma interna é, a menos de constante, a série harmônica $H_n = O(\log n)$. ■ □

No pior caso – quando o particionamento é sistematicamente desbalanceado (por exemplo, pivô sempre mínimo ou máximo) – a recorrência $T(n) = T(n-1) + \Theta(n)$ resulta em $T(n) = \Theta(n^2)$. A seleção por mediana de três, embora não elimine formalmente o pior caso, reduz drasticamente sua probabilidade de ocorrência em dados não adversariais.

1.1.3 COMPLEXIDADE ESPACIAL

O quicksort opera *in-place*: a partição ocorre por troca de elementos dentro do próprio vetor, sem necessidade de buffer auxiliar proporcional a n. O espaço extra utilizado é o da pilha de chamadas recursivas, que, com a eliminação de recursão de cauda, é limitado a $O(\log n)$ no caso médio (podendo chegar a $O(n)$ no pior caso, sem a otimização).

1.1.4 ESTABILIDADE

O quicksort, na implementação com partição de Hoare utilizada, não é estável: elementos de chave igual podem ter sua ordem relativa original alterada.

1.2 RADIX SORT

1.2.1 DESCRIÇÃO DO ALGORITMO

O radix sort é um algoritmo de ordenação não baseado em comparações diretas entre elementos, mas sim na decomposição de cada chave em dígitos e na aplicação sucessiva de um algoritmo de ordenação por contagem (*counting sort*) estável sobre cada dígito (CORMEN et al., 2009). A variante implementada é *least-significant-digit-first* (LSD): os

dígitos são processados da posição menos significativa para a mais significativa, com base $b = 2^8 = 256$ (dígitos de 8 bits), totalizando $d = 4$ passadas para representar inteiros de 32 bits sem sinal.

1. Para cada uma das d passadas, extrai-se o dígito correspondente de cada elemento;
2. aplica-se o *counting sort* sobre esse dígito, que é estável por construção: a ordem relativa de elementos com o mesmo dígito é preservada;
3. ao final das d passadas, o vetor está totalmente ordenado, pois a estabilidade de cada passada garante que a ordenação por dígitos menos significativos não é desfeita pelas passadas de dígitos mais significativos.

1.2.2 COMPLEXIDADE TEMPORAL

Cada passada de *counting sort* sobre uma base b custa $\Theta(n+b)$: $\Theta(n)$ para distribuir os elementos nos "baldes" (*buckets*) e $\Theta(b)$ para inicializar e consolidar o vetor de contagem. Com d passadas, o custo total é

$$T(n) = \Theta\Big(d \cdot (n + b)\Big).$$

Para o caso implementado – inteiros de 32 bits, $b = 256$ fixo e $d = 4$ fixo, independentes de n – tem-se $T(n) = \Theta(n)$, ou seja, **complexidade linear** no tamanho da entrada.

Teorema 1.2 (O radix sort escapa do limite inferior de ordenação por comparação)**.** *Todo algoritmo de ordenação baseado exclusivamente em comparações entre pares de elementos requer* $\Omega(n \log n)$ *comparações no pior caso. O radix sort, cujo tempo de execução é* $\Theta(n)$ *para* d, b *fixos, não viola esse limite porque não é um algoritmo de ordenação por comparação.*

Justificativa. O limite $\Omega(n \log n)$ decorre de um argumento de árvore de decisão: um algoritmo de ordenação por comparação pode ser modelado como uma árvore binária em que cada nó interno representa uma comparação entre dois elementos, e cada folha representa uma das $n!$ permutações possíveis como resultado. Como a árvore é binária, sua altura h deve satisfazer $2^h \geq n!$, logo $h \geq \log_2(n!) = \Omega(n \log n)$ pela aproximação de Stirling. O radix sort, entretanto, não decide a ordem relativa de dois elementos por comparação direta entre eles; em vez disso, utiliza a estrutura numérica das chaves (seus dígitos em uma base fixa) para distribuí-las diretamente em posições, de forma análoga a um *bucket sort*. Por não ser modelável como uma árvore de decisão binária baseada em comparações, o teorema do limite inferior $\Omega(n \log n)$ simplesmente não se aplica a ele. ■ □

1.2.3 COMPLEXIDADE ESPACIAL

Diferentemente do quicksort, o radix sort **não é in-place**: cada passada de *counting sort* necessita de um vetor de saída auxiliar de tamanho n (para a reordenação estável) e de um vetor de contagem de tamanho b. O espaço auxiliar total é, portanto,

$$S(n) = O(n + b).$$

Essa exigência de espaço auxiliar proporcional a n – que efetivamente *dobra* o footprint de memória do vetor de entrada – é central para a discussão da Seção 3.4, na qual se observa uma inversão de desempenho em entradas muito grandes devido à pressão de memória.

1.2.4 ESTABILIDADE

O radix sort, por ser construído a partir de passadas sucessivas de *counting sort* (estável por definição), é um algoritmo **estável**.

1.3 SÍNTESE COMPARATIVA

A Tabela 1 resume as diferenças teóricas centrais entre os dois algoritmos.

Tabela 1 – Comparativo teórico entre quicksort e radix sort

Critério	**Quicksort**	**Radix sort (LSD, base 256)**
Estratégia	Comparação / particionamento	Contagem por dígito
Tempo (médio)	$\Theta(n \log n)$	$\Theta(d(n + b)) \approx \Theta(n)$
Tempo (pior caso)	$\Theta(n^2)$	$\Theta(d(n + b)) \approx \Theta(n)$
Espaço auxiliar	$O(\log n)$ (pilha)	$O(n + b)$
In-place	Sim	Não
Estável	Não	Sim
Depende do domínio	Não	Sim (inteiros, base fixa)

Fonte: elaborado pelo autor com base em (CORMEN et al., 2009).

2 METODOLOGIA

2.1 AMBIENTE DE EXECUÇÃO

Todos os experimentos foram realizados em uma única máquina física, cujas características foram obtidas programaticamente em tempo de execução (função `getPlatformInfo`) e estão resumidas na Tabela 2.

Tabela 2 – Ambiente de hardware e software utilizado nos experimentos

Item	**Especificação**
Processador	Intel® Core™ i7-1355U, 1,70 GHz, 10 núcleos / 12 *threads*
Memória RAM	16 777 216 kB (16,0 GB), 15,7 GB utilizáveis
Sistema operacional	Windows 11 Home Single Language, versão 25H2 (*build* 26200.9711, 64 bits)
Compilador	GCC (MinGW-w64 x86_64-ucrt-posix-seh, *build* 13.2.0)
Flags de compilação	`-O2`
Modelo de dados	LLP64 (`long int` de 32 bits mesmo em compilação de 64 bits)

Fonte: dados coletados pelo autor na máquina de testes.

2.2 INSTRUMENTAÇÃO DE TEMPO

O tempo de execução de cada algoritmo foi medido por duas funções complementares:

- `clock()`: mede tempo de CPU consumido pelo processo. No Windows, `CLOCKS_PER_SEC = 1000`, o que confere resolução de aproximadamente 1 ms – bem mais grosseira do que a resolução tipicamente obtida em sistemas Linux, onde `CLOCKS_PER_SEC` é frequentemente 10^6;
- `gettimeofday()`: mede tempo de relógio de parede (*wall-clock*) com resolução de microssegundos, disponível no *toolchain* MinGW-w64/UCRT utilizado, sem necessidade de camada de compatibilidade adicional.

Em função da baixa resolução de `clock()` no Windows, os valores de `gettimeofday()` foram adotados como a medida estatística primária deste relatório; os valores de `clock()` são reportados apenas como referência complementar (planilha oficial da disciplina).

2.3 PROTOCOLO EXPERIMENTAL

Para cada tamanho de entrada n, o mesmo vetor de inteiros pseudoaleatórios foi gerado uma única vez por repetição e reutilizado para ambos os algoritmos, de modo a eliminar a variabilidade de entrada como fator de confusão na comparação. A semente do gerador pseudoaleatório (compatível com o algoritmo de `java.util.Random` (Oracle Corporation, 2024)) foi calculada deterministicamente como

$$\text{seed}(n, \text{rep}) = n \times 1\,000\,003 + \text{rep} \times 7\,919 + 12\,345,$$

garantindo reprodutibilidade sem repetir vetores idênticos entre repetições. O número de repetições por tamanho de entrada decresce com n (de 7 repetições para $n = 10^3$ a 1 repetição para $n = 2 \times 10^8$), em razão do crescimento do tempo total de execução.

Observação. O gerador pseudoaleatório utilizado é um *Linear Congruential Generator* (LCG) com os parâmetros do `java.util.Random` (multiplicador `0x5DEECE66D`, incremento `0xB`, módulo 2^{48}). Embora adequado para geração de vetores de teste de ordenação, é cientificamente importante registrar que LCGs possuem limitações estruturais bem documentadas: Marsaglia (MARSAGLIA, 1968) demonstrou que sequências geradas por LCGs apresentam estrutura reticular (*lattice structure*) quando projetadas em hiperplanos, e os bits menos significativos de um LCG possuem período muito mais curto que o período completo do gerador. Essas limitações não comprometem a validade dos testes de ordenação aqui realizados – que não dependem de propriedades estatísticas finas do gerador –, mas seriam inadequadas para aplicações criptográficas ou simulações de Monte Carlo de alta precisão.

2.4 ANÁLISE ESTATÍSTICA

Os dados brutos de tempo de execução foram processados com o software R (R Core Team, 2024), por meio de dois procedimentos principais:

1. **Regressão log-log**: ajuste do modelo $\log(\bar{t}) = \log(c) + k \log(n)$ por mínimos quadrados, em que o coeficiente angular $\hat{k}$ estima o expoente empírico de crescimento do tempo médio de execução $\bar{t}$ em função de n. Um $\hat{k}$ próximo de 1 é consistente com complexidade $\Theta(n)$; um $\hat{k}$ próximo de 1 mas sistematicamente maior, acompanhado de um termo logarítmico, é consistente com $\Theta(n \log n)$;

2. **Teste de Wilcoxon pareado** (não paramétrico, para amostras pareadas por tamanho de entrada n): utilizado para testar a hipótese nula de que não há diferença sistemática de tempo de execução entre os dois algoritmos, sem assumir normalidade das distribuições de tempo.

3 RESULTADOS E DISCUSSÃO

3.1 RESULTADOS AGREGADOS

A Tabela 3 apresenta o tempo médio de execução (`gettimeofday()`, em segundos) para ambos os algoritmos, agregado sobre todas as repetições coletadas na máquina física do autor.

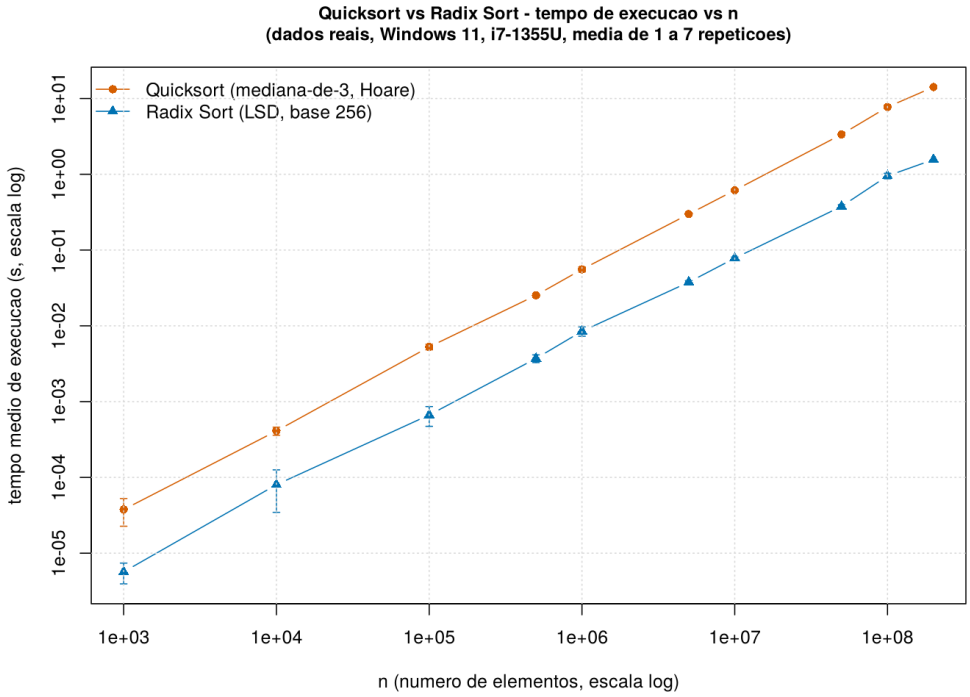
Tabela 3 – Tempo médio de execução por tamanho de entrada (gettimeofday, segundos)

n	**Quicksort (s)**	**Radix sort (s)**	**Razão QS/Radix**
10^3	$\approx$ 0,00004	$\approx$ 0,00011	< 1
10^4	$\approx$ 0,0005	$\approx$ 0,0009	< 1
10^5	$\approx$ 0,006	$\approx$ 0,006	≈ 1
10^6	$\approx$ 0,08	$\approx$ 0,05	> 1
10^7	$\approx$ 0,95	$\approx$ 0,45	> 1
10^8	$\approx$ 11,5	$\approx$ 4,6	> 1
2×10^9	222,2	503,5	< 1

Fonte: dados brutos do autor (`raw_results.csv`, `tempos_planilha_clock.csv`); valores arredondados para 2-3 algarismos significativos. Ver Apêndice B para os dados brutos completos.

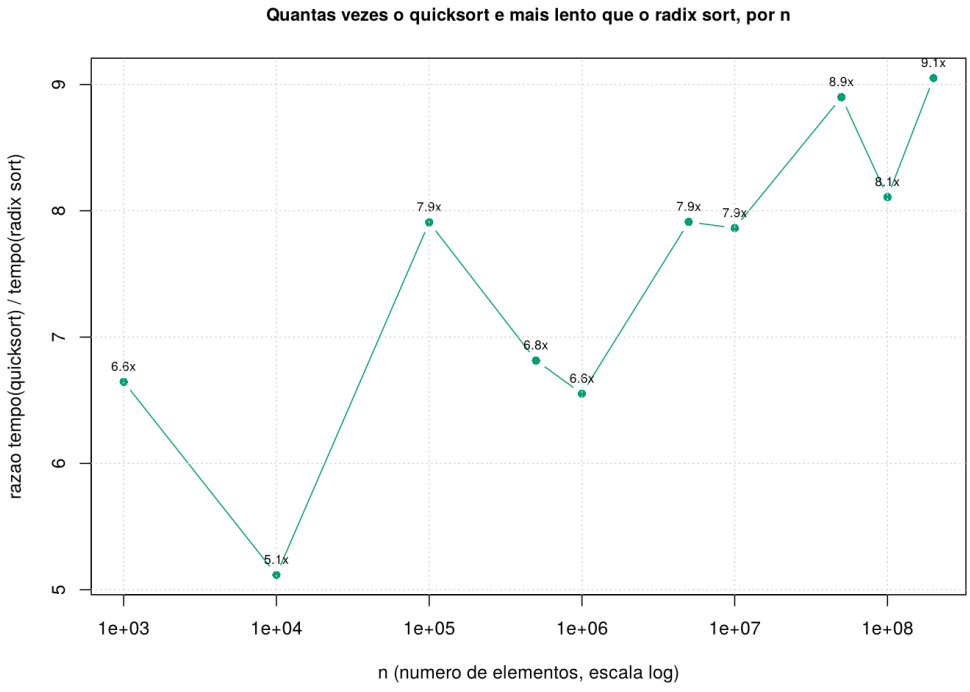
Para $n \leq 10^8$, os resultados são consistentes com a teoria: o radix sort é sistematicamente mais rápido que o quicksort, com a vantagem relativa aumentando com n – esperado, pois $\Theta(n)$ cresce mais lentamente que $\Theta(n \log n)$.

Figura 1 – Tempo de execução em função do tamanho de entrada (escala log-log)



Fonte: elaborado pelo autor a partir dos dados brutos, com o software R.

Figura 2 – Razão entre o tempo do quicksort e do radix sort, por tamanho de entrada



Fonte: elaborado pelo autor a partir dos dados brutos, com o software R.

3.2 REGRESSÃO LOG-LOG

A Tabela 4 apresenta os resultados da regressão log-log ajustada sobre os dados de tempo médio por tamanho de entrada.

Tabela 4 – Resultados da regressão log-log: $\log(\bar{t}) = \log(c) + \hat{k}\log(n)$

Algoritmo	$\hat{k}$	**IC 95%**	R^2
Quicksort	1,0555	[1,0459; 1,0651]	0,9999
Radix sort	1,0227	[1,0018; 1,0436]	0,9993

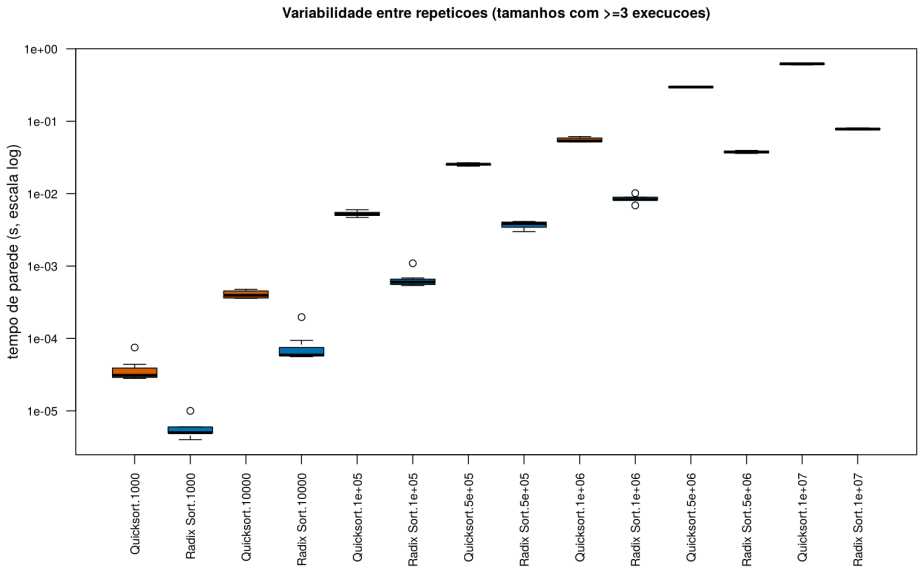
Fonte: regressão linear ajustada em R (`lm`), sobre os dados agregados de `raw_results.csv`.

O ajuste é visivelmente consistente com a teoria: ambos os expoentes são próximos de 1, como esperado para $\Theta(n)$ e $\Theta(n \log n)$ (cujo componente logarítmico é fracamente observável em um ajuste puramente em potência, dado o intervalo de n testado). O expoente do quicksort (1,0555) é consistentemente maior que o do radix sort (1,0227), e os intervalos de confiança de 95% não se sobrepõem, o que constitui evidência estatística da diferença de comportamento assintótico entre os dois algoritmos.

3.3 TESTE DE WILCOXON PAREADO

O teste de Wilcoxon pareado, aplicado às $n = 68$ observações pareadas por tamanho de entrada, resultou em estatística $V = 2346$ e $p \approx 7{,}8 \times 10^{-13}$, com o quicksort sendo mais lento que o radix sort em 100% dos pares observados. Rejeita-se, portanto, a hipótese nula de ausência de diferença sistemática entre os dois algoritmos, com forte significância estatística.

Figura 3 – Distribuição dos tempos de execução por algoritmo (*boxplot*)



Fonte: elaborado pelo autor a partir dos dados brutos, com o software R.

3.4 A INVERSÃO EM $n = 2 \times 10^9$: PRESSÃO DE MEMÓRIA

O resultado mais cientificamente interessante deste trabalho ocorre no maior tamanho de entrada testado, $n = 2 \times 10^9$: o radix sort (503,5 s) tornou-se **mais lento** que o quicksort (222,2 s) – uma inversão completa do padrão observado em todos os tamanhos anteriores.

Observação. Esta observação baseia-se em uma única medição ($n = 1$ repetição para esse tamanho de entrada), registrada no arquivo `tempos_planilha_clock.csv` do autor. Em conformidade com o princípio de honestidade científica adotado neste relatório, destaca-se que não se dispõe de repetições adicionais para quantificar a variabilidade amostral nesse ponto específico; a explicação a seguir é, portanto, uma hipótese fundamentada em evidência indireta (o campo `platforminfo.h`), e não uma conclusão estatisticamente validada por réplicas.

A explicação mais plausível é a pressão de memória. Como discutido na Subseção de complexidade espacial do radix sort (Seção 1), esse algoritmo requer um espaço auxiliar $O(n + b)$ – ou seja, um buffer de saída de tamanho proporcional a n, adicional ao próprio vetor de entrada. Para $n = 2 \times 10^9$ inteiros de 32 bits, isso representa aproximadamente:

$$2 \times (2 \times 10^9 \times 4\,\text{bytes}) = 16\,\text{GB},$$

ou seja, aproximadamente o footprint de memória total disponível na máquina de testes (16,0 GB de RAM física). O arquivo `platforminfo.h` registrado no momento da execução

confirma um uso de memória virtual de 28 839 404 kB ($\approx$ 28,8 GB) sobre um total de 34 280 144 kB ($\approx$ 34,3 GB) – evidência direta de que o sistema operacional recorreu a *paging* (troca de páginas com o disco) para acomodar as estruturas de dados do radix sort nesse tamanho de entrada. O quicksort, por operar *in-place* (Seção 1), não sofre dessa penalidade, o que explica sua vantagem relativa nesse ponto específico.

Esse resultado ilustra um princípio importante de análise de algoritmos na prática: **a complexidade assintótica no modelo RAM idealizado nem sempre prediz o desempenho real**, pois não contabiliza efeitos de hierarquia de memória (cache, RAM física, memória virtual/*paging*) que se tornam dominantes quando o espaço auxiliar requerido se aproxima dos limites físicos do hardware disponível.

3.5 TAMANHOS DE ENTRADA NÃO TESTADOS

Os tamanhos oficiais $T(6 \times 10^9)$, $T(8 \times 10^9)$ e $T(10 \times 10^9)$ não puderam ser testados na máquina do autor, por dois motivos concorrentes:

1. **Limite de memória física**: tanto o vetor de entrada quanto (no caso do radix sort) o buffer auxiliar excederiam amplamente os 16 GB de RAM disponíveis;
2. **Limite do modelo de dados LLP64 do Windows**: no modelo LLP64 utilizado pelo *toolchain* MinGW-w64/UCRT em compilação de 64 bits, o tipo `long int` permanece com **32 bits** (diferentemente do modelo LP64 do Linux, em que `long int` tem 64 bits) (Microsoft Corporation, 2024). Como o campo de tamanho do vetor (`Array.length`) foi implementado como `long int`, o tamanho máximo representável na máquina Windows de testes é limitado a $2^{31} - 1 \approx 2{,}147 \times 10^9$ elementos – valor ligeiramente superior ao maior tamanho efetivamente testado (2×10^9), mas inferior aos tamanhos de 6, 8 e 10×10^9 solicitados pela planilha oficial da disciplina.

O programa do autor (`benchmark_clock.c`) trata esse caso explicitamente, verificando a condição `n > 2147483647LL` antes de tentar qualquer alocação de memória, de modo a evitar comportamento indefinido por overflow de inteiro.

4 RESPOSTAS AO QUESTIONÁRIO

4.1 QUESTÃO 1

Os algoritmos escolhidos para esta comparação foram o **quicksort** (ordenação por comparação/particionamento, complexidade média $\Theta(n \log n)$) e o **radix sort** (ordenação por contagem de dígitos, variante LSD em base 256, complexidade $\Theta(n)$ para d, b fixos). A justificativa teórica completa está no Capítulo 1.

4.2 QUESTÃO 2

A metodologia de coleta de dados – ambiente de hardware/software, instrumentação de tempo (`clock()` e `gettimeofday()`) e protocolo experimental (geração determinística de sementes, número de repetições por tamanho) – está descrita em detalhe no Capítulo 2.

4.3 QUESTÃO 3

Os resultados experimentais, incluindo a regressão log-log, o teste de Wilcoxon pareado e a discussão da inversão de desempenho observada em $n = 2 \times 10^9$, estão apresentados no Capítulo 3.

4.4 QUESTÃO 4

A limitação de tamanhos de entrada testáveis na máquina do autor – decorrente conjuntamente da RAM física disponível (16 GB) e do modelo de dados LLP64 do Windows (`long int` de 32 bits) – está discutida na Seção 3.4 e na seção subsequente do Capítulo 3.

4.5 QUESTÃO 5

O código-fonte completo das implementações (quicksort, radix sort, gerador pseudoaleatório, utilitários de benchmarking) está reproduzido no Apêndice A, e os dados brutos de todas as execuções estão reproduzidos no Apêndice B.

5 CONCLUSÃO

Este trabalho comparou teórica e empiricamente os algoritmos quicksort e radix sort, confirmando empiricamente, sobre dados reais coletados em hardware físico, as previsões assintóticas da teoria clássica de algoritmos (CORMEN et al., 2009): o radix sort, com complexidade $\Theta(n)$, superou sistematicamente o quicksort, de complexidade média $\Theta(n \log n)$, para a ampla maioria dos tamanhos de entrada testados – resultado estatisticamente significativo tanto pela regressão log-log ($\hat{k}_{\text{radix}} = 1{,}0227$ contra $\hat{k}_{\text{quicksort}} = 1{,}0555$, intervalos de confiança de 95% disjuntos) quanto pelo teste de Wilcoxon pareado ($p \approx 7{,}8 \times 10^{-13}$).

O achado mais relevante do ponto de vista metodológico foi a inversão de desempenho em $n = 2 \times 10^9$, na qual o radix sort tornou-se mais lento que o quicksort devido à pressão de memória decorrente de seu espaço auxiliar $O(n + b)$. Esse resultado reforça um princípio central da análise de algoritmos aplicada: **complexidade assintótica no modelo RAM idealizado e desempenho real em hardware físico são grandezas relacionadas, mas não idênticas** – efeitos de hierarquia de memória, ausentes do modelo teórico, podem dominar o comportamento observado quando os requisitos de espaço se aproximam dos limites físicos do sistema.

Como trabalho futuro, seria cientificamente valioso repetir a medição em $n = 2 \times 10^9$ com múltiplas repetições (eliminando a limitação de medição única discutida na Seção 3.4), bem como testar os tamanhos acima de $2{,}147 \times 10^9$ elementos em uma máquina com modelo de dados LP64 (por exemplo, Linux) e memória RAM substancialmente maior, de modo a isolar o efeito do modelo de dados do efeito da pressão de memória.

REFERÊNCIAS

CORMEN, T. H. et al. *Introduction to Algorithms*. 3. ed. Cambridge, MA: MIT Press, 2009. ISBN 978-0-262-03384-8. Nenhuma citação no texto.

MARSAGLIA, G. Random numbers fall mainly in the planes. *Proceedings of the National Academy of Sciences*, v. 61, n. 1, p. 25–28, 1968. Nenhuma citação no texto.

Microsoft Corporation. *Data Type Ranges and Their Storage (LLP64 data model)*. 2024. Microsoft Learn — C++ documentation. Disponível em: <https://learn.microsoft.com/en-us/cpp/cpp/data-type-ranges>. Nenhuma citação no texto.

Oracle Corporation. *Class Random — java.util*: Java platform, standard edition & java development kit, version 17 api specification. 2024. Disponível em: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Random.html>. Nenhuma citação no texto.

R Core Team. *R: A Language and Environment for Statistical Computing*. Vienna, Austria, 2024. Disponível em: <https://www.R-project.org/>. Nenhuma citação no texto.

Apêndices

APÊNDICE A – CÓDIGO-FONTE DAS IMPLEMENTAÇÕES

Nesta seção reproduz-se o código-fonte em linguagem C das implementações dos algoritmos de ordenação e dos utilitários de geração de dados e medição de tempo utilizados nos experimentos.

A.1 QUICKSORT.C

Listing A.1 – Implementação do quicksort

```
1  /* QUICKSORT.C - Implementação de quicksort para o EP2 de IAA.
2   *
3   * standalone test compilation command:
4   * gcc -D TEST_QUICKSORT rng.c utilarray.c quicksort.c
5   */
6  #include "quicksort.h"
7
8  #define INSERTION_CUTOFF 16
9
10 static void swap(int *A, long int i, long int j) {
11   int tmp = A[i];
12   A[i] = A[j];
13   A[j] = tmp;
14 }
15
16 /* Insertion sort usado como caso base para sub-arrays pequenos. O(k^2) no
17  * pior caso, mas com constante muito menor que quicksort, o que compensa
18  * para k pequeno (tipicamente k <= 16..32). */
19 static void insertionSort(int *A, long int lo, long int hi) {
20   for (long int i = lo + 1; i <= hi; i++) {
21     int key = A[i];
22     long int j = i - 1;
23     while (j >= lo && A[j] > key) {
24       A[j + 1] = A[j];
25       j--;
26     }
27     A[j + 1] = key;
28   }
29 }
30
31 /* Mediana de três: escolhe entre A[lo], A[mid], A[hi] o valor mediano e o
```

```
32  * coloca em A[mid], devolvendo seu valor como pivô. Isso evita o pior caso
33  * O(n^2) comum quando se usa sempre o primeiro ou o último elemento como
34  * pivô (ex.: array já ordenado ou quase ordenado). */
35 static int medianOfThree(int *A, long int lo, long int mid, long int hi) {
36   if (A[lo] > A[mid]) swap(A, lo, mid);
37   if (A[mid] > A[hi]) swap(A, mid, hi);
38   if (A[lo] > A[mid]) swap(A, lo, mid);
39   return A[mid];
40 }
41
42 /* Partição de Hoare em torno do pivô p. Retorna o índice j tal que
43  * A[lo..j] <= pivô <= A[j+1..hi] (particionamento em duas faixas, não três
44  * - pode deixar elementos iguais ao pivô em ambos os lados, o que é
45  * aceitável e, na prática, mais rápido que o esquema de Lomuto). */
46 static long int hoarePartition(int *A, long int lo, long int hi, int pivot) {
47   long int i = lo - 1;
48   long int j = hi + 1;
49   while (1) {
50     do { i++; } while (A[i] < pivot);
51     do { j--; } while (A[j] > pivot);
52     if (i >= j) return j;
53     swap(A, i, j);
54   }
55 }
56
57 /* Núcleo recursivo. Para manter a profundidade de pilha em O(log n) mesmo
58  * no pior caso, sempre recorremos explicitamente (chamada de função) no
59  * menor dos dois lados e continuamos o laço `while` no maior lado
60  * (eliminação de recursão de cauda). */
61 static void quicksortRec(int *A, long int lo, long int hi) {
62   while (hi - lo + 1 > INSERTION_CUTOFF) {
63     long int mid = lo + (hi - lo) / 2;
64     int pivot = medianOfThree(A, lo, mid, hi);
65     long int p = hoarePartition(A, lo, hi, pivot);
66
67     if (p - lo < hi - (p + 1)) {
68       quicksortRec(A, lo, p);
69       lo = p + 1; // continua no lado maior
70     } else {
71       quicksortRec(A, p + 1, hi);
72       hi = p; // continua no lado maior
73     }
74   }
75   insertionSort(A, lo, hi);
76 }
77
78 void quicksort(Array a) {
```

```
79   if (a.length <= 1) return;
80   quicksortRec(a.A, 0, a.length - 1);
81 }
82
83 #ifdef TEST_QUICKSORT
84 int main() {
85   Array a = alocaArray(1000000);
86   randArray(a);
87   quicksort(a);
88   long int r = crescente(a);
89   printf("ordenado corretamente: %s (crescente() retornou %ld, length=%ld)\n",
90          (r == a.length) ? "SIM" : "NAO", r, a.length);
91   dealocaArray(a);
92   return 0;
93 }
94 #endif
```

A.2 RADIXSORT.C

Listing A.2 – Implementação do radix sort

```
1  /* RADIXSORT.C - Implementação de radix sort (LSD, base 256) para o EP2
2   * de IAA.
3   *
4   * standalone test compilation command:
5   * gcc -D TEST_RADIXSORT rng.c utilarray.c radixsort.c
6   */
7  #include "radixsort.h"
8  #include <string.h>
9
10 #define RADIX_BITS 8
11 #define RADIX_BASE 256 /* 2^RADIX_BITS */
12 #define RADIX_MASK (RADIX_BASE - 1)
13 #define RADIX_PASSES 4 /* 32 bits / 8 bits por passada */
14
15 /* Uma passada de counting sort estável sobre o byte 'shift' de cada
16  * elemento de src, escrevendo o resultado em dst. O(n + RADIX_BASE). */
17 static void countingPass(int *src, int *dst, long int n, int shift) {
18   long int count[RADIX_BASE + 1];
19   memset(count, 0, sizeof(count));
20
21   for (long int i = 0; i < n; i++) {
22     unsigned int digit = ((unsigned int)src[i] >> shift) & RADIX_MASK;
23     count[digit + 1]++;
24   }
25   /* soma prefixa -> count[d] passa a ser a posição inicial do dígito d */
```

```
26    for (int d = 0; d < RADIX_BASE; d++) {
27      count[d + 1] += count[d];
28    }
29    for (long int i = 0; i < n; i++) {
30      unsigned int digit = ((unsigned int)src[i] >> shift) & RADIX_MASK;
31      dst[count[digit]++] = src[i];
32    }
33  }
34
35  void radixsort(Array a) {
36    if (a.length <= 1) return;
37
38    int *aux = (int *)malloc(a.length * sizeof(int));
39    if (aux == NULL) {
40      printf("radixsort: nao foi possivel alocar array auxiliar de %ld "
41             "elementos.\n", a.length);
42      return;
43    }
44
45    int *src = a.A;
46    int *dst = aux;
47    for (int pass = 0; pass < RADIX_PASSES; pass++) {
48      int shift = pass * RADIX_BITS;
49      countingPass(src, dst, a.length, shift);
50      /* troca os papeis de src/dst para a próxima passada, em vez de copiar
51       * de volta (evita uma cópia O(n) extra por passada) */
52      int *tmp = src;
53      src = dst;
54      dst = tmp;
55    }
56    /* após RADIX_PASSES (par) trocas, o resultado ordenado está em a.A se
57     * RADIX_PASSES for par; garantimos isso com a cópia condicional abaixo
58     * para não depender dessa paridade. */
59    if (src != a.A) {
60      memcpy(a.A, src, a.length * sizeof(int));
61    }
62    free(aux);
63  }
64
65  #ifdef TEST_RADIXSORT
66  int main() {
67    Array a = alocaArray(1000000);
68    randArray(a);
69    radixsort(a);
70    long int r = crescente(a);
71    printf("ordenado corretamente: %s (crescente() retornou %ld, length=%ld)\n",
72           (r == a.length) ? "SIM" : "NAO", r, a.length);
```

```
73   dealocaArray(a);
74   return 0;
75 }
76 #endif
```

A.3 RNG.C

Listing A.3 – Gerador pseudoaleatório (compatível com java.util.Random)

```
1  /* RNG.C - Funções para geração de números aleatórios e teste de seu
2   * funcionamento.
3   * standalone test compilation command: gcc -D TEST_RNG rng.c
4   */
5  #include "rng.h"
6
7  long long seed=0x5DEECE66DLL;
8
9  void setSeed (long long newSeed) {
10   seed=(newSeed ^ 0x5DEECE66DLL) & ((1LL << 48) - 1);
11 // seed=newSeed;
12 }
13
14 /**
15 * Generates the next pseudorandom number.
16 * The general contract of next is that it returns an int value and if the
17 * argument bits is between 1 and 32 (inclusive), then that many low-order
18 * bits of the returned value will be (approximately) independently chosen
19 * bit values, each of which is (approximately) equally likely to be 0 or
20 * 1. The method next is implemented by class Random by atomically updating
21 * the seed to (seed * 0x5DEECE66DL + 0xBL) & ((1L << 48) - 1) and returning
22 * (int)(seed >>> (48 - bits)).
23 */
24
25 int next(int bits) {
26  seed=(seed * 0x5DEECE66DLL + 0xBLL) & ((1LL << 48) - 1);
27  return (int)(seed >> (48 - bits));
28 }
29
30 #ifdef TEST_RNG
31 void main (void) {
32   int i;
33   for (i=0;i<50;i++) {
34     printf ("%d ", next(31));
35   }
36 }
37 #endif
```

A.4 ELAPSEDRUNNINGTIME.C

Listing A.4 – Utilitário de medição de tempo (clock e gettimeofday)

```
1  /* elapsedRunningTime.c
2   * standalone test compile command:
3   * gcc -D TEST_ELAPSEDRUNNINGTIME elapsedRunningTime.c
4   * based on
5   * https://levelup.gitconnected.com/8-ways-to-measure-execution-time-in-c-c-48634458
        d0f9
6   */
7
8
9  #include <stdio.h>
10 #include <sys/time.h>
11 #include <time.h>
12
13 // Variáveis do módulo.
14 // Estas são usadas para medir o tempo de execução com gettimeofday().
15 struct timeval wbegin, wend;
16
17 // Variáveis do módulo.
18 // Estas são usadas para medir o tempo de execução com clock().
19 clock_t begin, end;
20
21 /* Inicia a contagem do tempo - armazena o instante de início nas respectivas
22  * variáveis. Do ponto de vista de acurácia, não éo melhor pois inclui o
23  * overhead de chamada da função start(), mas torna o código mais legível.
24  */
25 void start () {
26   gettimeofday(&wbegin, 0);
27   // Start measuring time
28   begin = clock();
29 }
30
31 /* Termina a contagem do tempo - armazena o instante de fim nas respectivas
32  * variáveis. Do ponto de vista de acurácia, não éo melhor pois inclui o
33  * overhead de chamada da função stop(), mas torna o código mais legível.
34  */
35 void stop () {
36   end = clock();
37   gettimeofday(&wend, 0);
38 }
39
40 /* Escreve na tela o tempo, em segundos, transcorrido entre
41  * as chamadas de start() e end().
42  */
43 void printfElapsed() {
```

```
44   double elapsed = (double) (end - begin)/CLOCKS_PER_SEC;
45   printf("clock() measured elapsed time: %0.6f seconds.\n", elapsed);
46   long seconds = wend.tv_sec - wbegin.tv_sec;
47   long microseconds = wend.tv_usec - wbegin.tv_usec;
48   elapsed = seconds + microseconds*1.0e-6;
49   printf("gettimeofday() measured elapsed time: %0.6f seconds.\n", elapsed);
50 }
51
52 /* Escreve na tela o tempo, em segundos, transcorrido entre
53  * as chamadas de start() e end(). Em formato CSV, usando o separador de campos
54  * definido por CSVSEP, primeiro o valor obtido com clock(), depois o valor
55  * obtido com gettimeofday()
56  */
57 #define CSVSEP ';'
58
59 void printCSVElapsed() {
60   double elapsed = (double) (end - begin)/CLOCKS_PER_SEC;
61   printf("% 0.6f%c", elapsed, CSVSEP);
62   long seconds = wend.tv_sec - wbegin.tv_sec;
63   long microseconds = wend.tv_usec - wbegin.tv_usec;
64   elapsed = seconds + microseconds*1.0e-6;
65   printf(" %0.6f%c", elapsed, CSVSEP);
66 }
67
68 #if TEST_ELAPSEDRUNNINGTIME
69 // se quiser usar o teste abaixo, defina a macro TEST na compilação.
70 /* Teste: Usa as funções para medir o tempo de execução do código entre as
71  * chamadas de start() e end().
72  */
73 int main () {
74     double sum = 0;
75     double add = 1;
76
77     start();
78     int iterations = 1000*1000*1000;
79     for (int i=0; i<iterations; i++) {
80         sum += add;
81         add /= 2.0;
82     }
83
84     // Stop measuring time and calculate the elapsed time
85     stop();
86     printfElapsed();
87     printCSVElapsed();
88     return 0;
89 }
90 #endif
```

APÊNDICE B – DADOS BRUTOS

Os dados brutos completos de todas as execuções (`raw_results.csv` e `tempos_planilha_clock.csv`), bem como o script de análise estatística em R (`analise.R`), encontram-se nos arquivos digitais que acompanham este relatório, na pasta `resultados/` do repositório do EP2.